# Portkey AI Tutorial for test_Portkey.py (NYU Gateway)

Updated: 2026-04-21

## Quick Start

1) Connect to NYU VPN.

2) Set PORTKEY_API_KEY in your terminal session.

3) Select one model in the request and run the script.

## Step 1. Connect to NYU VPN

Connect to NYU VPN before sending requests to the NYU gateway URL:

https://ai-gateway.apps.cloud.rt.nyu.edu/v1

## Step 2. Set Portkey AI API Key

Use environment variables instead of hardcoding keys:

```
export PORTKEY_API_KEY="your-actual-key"
python test_Portkey.py
```

## Step 3. Select Your Model

Choose one embedding model for embeddings or one general model for chat/completions.

## Model Summary (Reasoning Verified)

Verification basis: Google Gemini, OpenAI, and Anthropic model docs (checked on 2026-04-21).

| Model | Category | Reasoning Type | Verification Note |
|---|---|---|---|
| @vertexai/gemini-embedding-001 | Embedding | N/A | Google Gemini docs: Specialized task model for embeddings |
| @vertexai/gemini-2.5-pro | General | Reasoning | Google Gemini docs: 'deep |

| | | | |
|---|---|---|---|
| | | | reasoning and coding capabilities' |
| @vertexai/gemini-3.1-pro-preview | General | Reasoning | Google Gemini docs: 'advanced intelligence, complex problem-solving' |
| @vertexai/gemini-2.5-flash | General | Non-reasoning | Google Gemini docs: tasks that require reasoning |
| @o4-mini/o4-mini | General | Reasoning | OpenAI o-series family is reasoning-oriented |
| @gpt-4o/gpt-4o | General | Non-reasoning | OpenAI GPT-4o line is general multimodal |
| @gpt-4o-mini/gpt-4o-mini | General | Non-reasoning | OpenAI GPT-4o mini line is fast general model |
| @vertexai/gemini-2.5-flash-lite | General | Non-reasoning | Google Gemini docs: fastest/budget-friendly, no explicit reasoning positioning |
| @vertexai/gemini-3.1-flash-lite-preview | General | Non-reasoning | Google Gemini docs: speed/cost-focused preview |
| @vertexai/gemini-3-flash-preview | General | Non-reasoning | Google Gemini docs: performance/cost-focused preview |
| @bedrock/us.anthropic.claude-sonnet-4-20250514-v1:0 | General | Reasoning | Anthropic docs: Sonnet has extended thinking support |
| @vertexai/anthropic.claude-sonnet-4-6 | General | Reasoning | Anthropic docs: speed + intelligence with thinking support |
| @vertexai/anthropic.claude-sonnet-4-5@20250929 | General | Reasoning | Anthropic docs: Sonnet family supports thinking modes |
| @vertexai/anthropic.claude-opus-4-6 | General | Reasoning | Anthropic docs: most capable for complex reasoning |

| | | | |
|---|---|---|---|
| @vertexai/anthropic.claude-opus-4-5@20251101 | General | Reasoning | Anthropic docs: Opus family for hardest reasoning tasks |
| @vertexai/anthropic.claude-haiku-4-5@20251001 | General | Non-reasoning | Anthropic docs: fast model with near-frontier intelligence |

## Appendix A. Full Script (test_Portkey.py)

Included from your current file, with API key redacted for security.

```
from portkey_ai import Portkey


portkey = Portkey(

  base_url = "https://ai-gateway.apps.cloud.rt.nyu.edu/v1",

  api_key = "<REDACTED_API_KEY>"

)


prompt = '''

Generate verilog code for a 4-bit carry select adder.

This is built using full-adders and muxes.

A 2-bit carry select adder uses 4 full-adders, 2 for a ripple carry adder with
carry in being 0 and 2 for a ripple carry adder with a carry in of 1.

We then select the correct S0 and S1 using a 2-to-1 MUX where we use carry in
as the select.

A 4-bit carry select adder should use 2-bit carry select adders to perform the
adder operation.

We do this using two 2-bit carry select adders where 1 does the operation for
the first 2 bits and the other does the operation for the last 2 bits.

The first 2 bit carry select adder obviously has a carry in of 0 but the carry
in of the second 2 bit carry select adder is the carry out of the first 2-bit
carry select adder.

Here are the parameters for the 2-bit ripple carry adder:

    - input [1:0] a,b

    - input cin
```

```
    - output [1:0] sum
    - output cout
Here are the parameters for the full-adder:
    - input a,b,cin
    - output sum, cout
Here are the parameters for the 2-bit ripple carry adder:
    - input [3:0] a,b
    - output [3:0] sum
    - output cout
'''


# embedding = portkey.embeddings.create(
#     model = "@vertexai/gemini-embedding-001",
#     input = "The food was delicious and the waiter...",
#     encoding_format = "float"
# )


# print(embedding)


response = portkey.chat.completions.create(
    # model = "@vertexai/gemini-2.5-pro",
    # model = "@bedrock/us.anthropic.claude-sonnet-4-20250514-v1:0",
    # model = "@o4-mini/o4-mini",
    # model = "@gpt-4o-mini/gpt-4o-mini",
    # model = "@gpt-4o/gpt-4o",
    # model = "@vertexai/gemini-3.1-pro-preview",
    # model = "@vertexai/gemini-3.1-flash-lite-preview",
    # model = "@vertexai/gemini-3-flash-preview",
    # model = "@vertexai/gemini-2.5-pro",
```

```
    # model = "@vertexai/gemini-2.5-flash-lite",
    # model = "@vertexai/gemini-2.5-flash",
    # model = "@vertexai/anthropic.claude-sonnet-4-6",
    # model = "@vertexai/anthropic.claude-sonnet-4-5@20250929",
    # model = "@vertexai/anthropic.claude-opus-4-6",
    # model = "@vertexai/anthropic.claude-opus-4-5@20251101",
    # model = "@vertexai/anthropic.claude-haiku-4-5@20251001",

    messages = [
      {"role": "system", "content": "You are a helpful assistant."},
      {"role": "user", "content": prompt}
    ],
    # max_tokens = 512
)

print(response.choices[0].message.content)
```

## Security Note

If a real API key was shared in plain text, rotate/revoke it immediately.